Cover Page

•

Mobile Forensics (MBF 712S) Lab Investigation	
Lab No:	*Lab 3*
Lead Investigator:	*Titus Shaangelo 222084774* *Barnabas iihuhwa 223081701* *Tjijenda.u 220113009*
Qualifications:	*B.COMP.SC. IN CYBER SECURITY*
Nature of Case: **(Lab Objective)**	The objective of this lab is to demonstrate the process of extracting and analysing data from an Android device. This is done by creating a virtual Android device and using forensic tools to explore system files and recover data that may be useful in an investigation.
Due Date:	*29 March 2026*

Description of Evidence		**Vendor Name**	**Model /Serial No.**
Item	**Description**	**Vendor Name**	**Model / Serial No.**
Item #1	Android Studio	Google	Latest Version
Item #2	Android Virtual Device (AVD)	Android SDK	Virtual Device
Item #3	Android Debug Bridge (ADB)	Google	Platform Tools
Item #4	Command Prompt	Microsoft	Windows OS
Item #5	cache.img file	Android System	N/A

1. Introduction

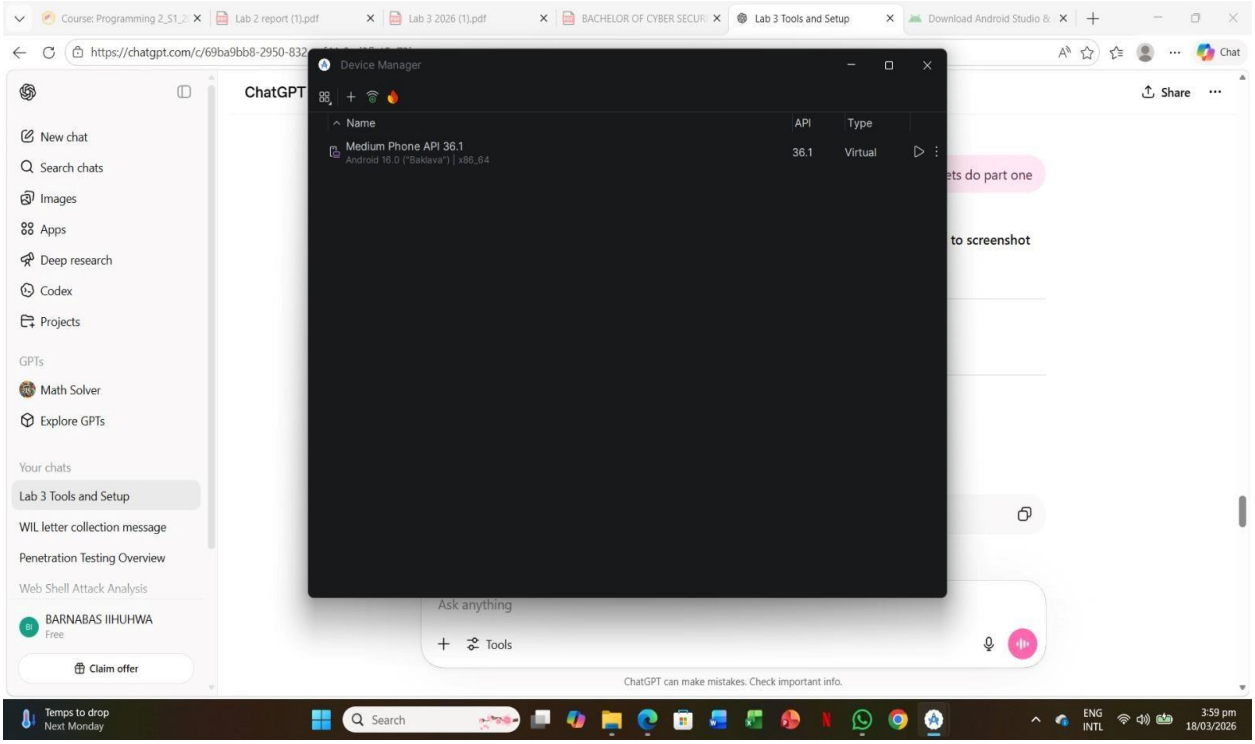
Mobile forensics is the process of collecting and analysing data from mobile devices for investigation purposes. In this lab, a virtual Android device was created using Android Studio, and the internal system was accessed using Android Debug Bridge. These tools allow investigators to explore files, recover data, and understand how information is stored on mobile devices.

2. Lab Procedures

Step 1: Opening Android Studio

Android Studio was successfully launched on the computer. From the welcome screen, the Virtual Device Manager was accessed by selecting the option:
More Actions → Virtual Device Manager. This allowed access to the environment used to create and manage virtual Android devices.

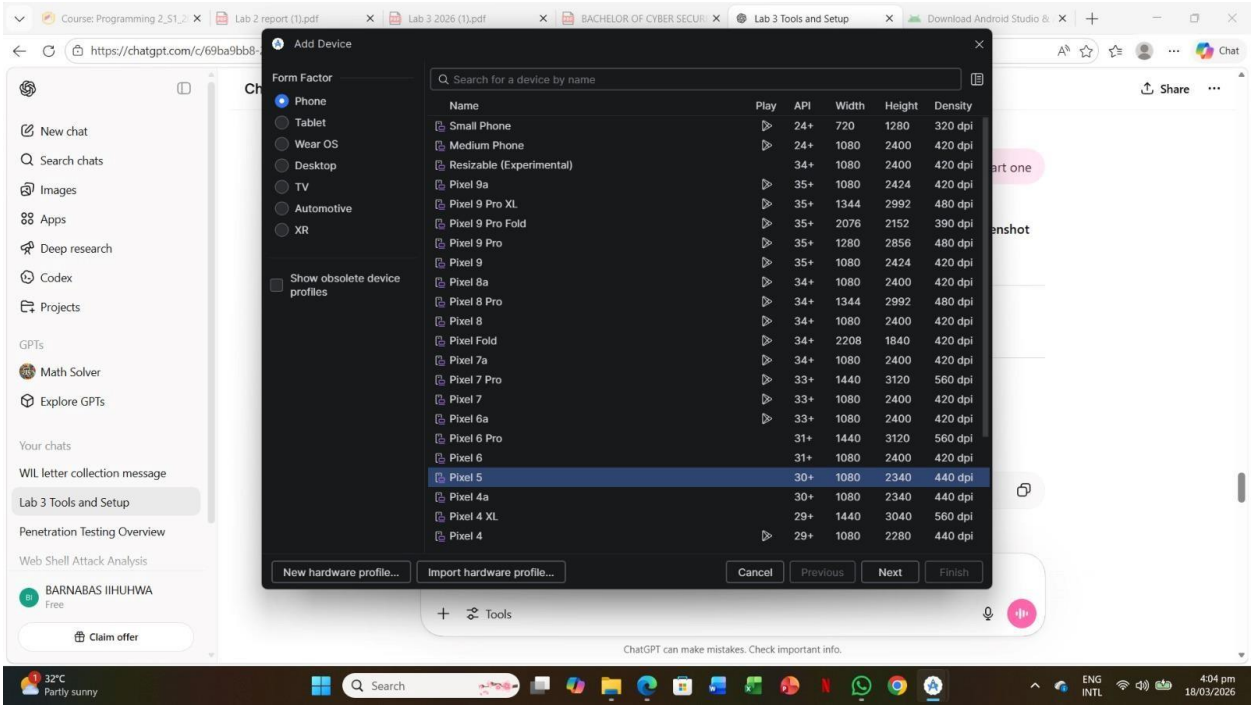
Screenshot Android Studio home screen or Virtual Device Manager open:



Step 2: Creating an Android Virtual Device (AVD)

A new Android Virtual Device was created by selecting the Create Virtual Device option. A suitable device model was chosen, for example Pixel 5, to simulate a real Android smartphone.

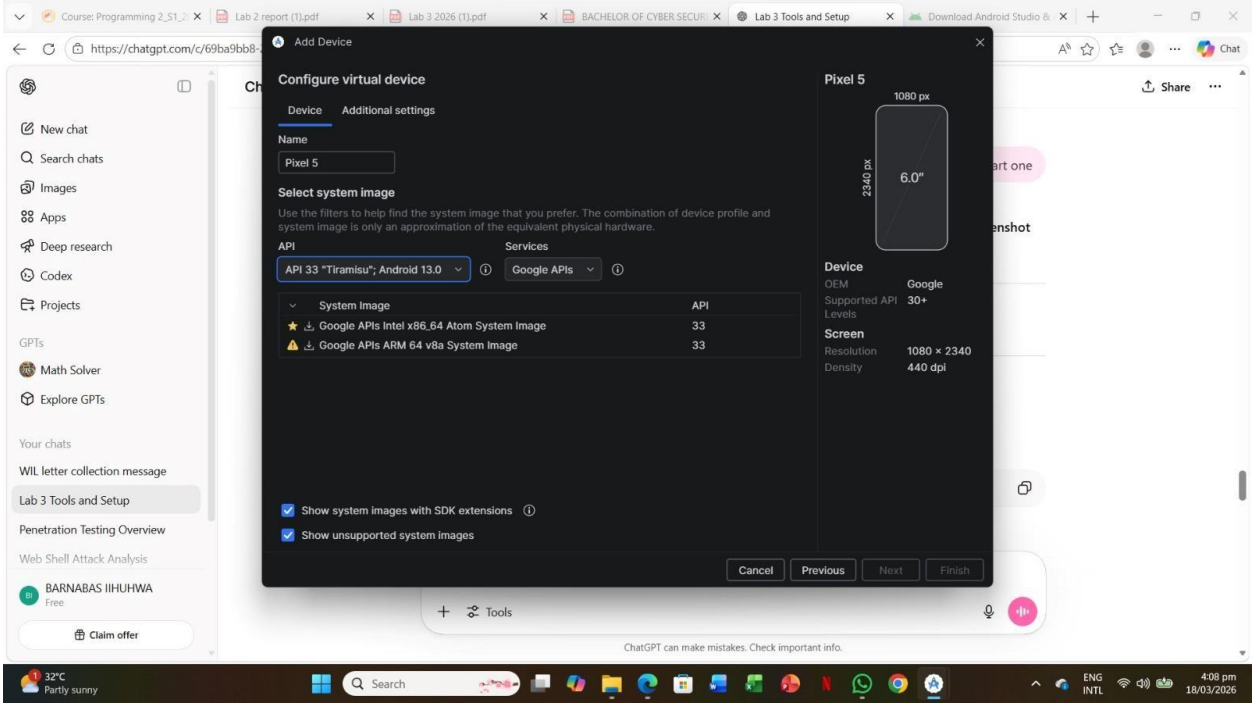
Insert Screenshot Device selection screen showing chosen phone



Step 3: Selecting the Android System Image

An Android system image was selected Android 13 / API Level 33. the system image was not previously installed, we downloaded it. After that, the setup was completed by clicking Next and then Finish**.**

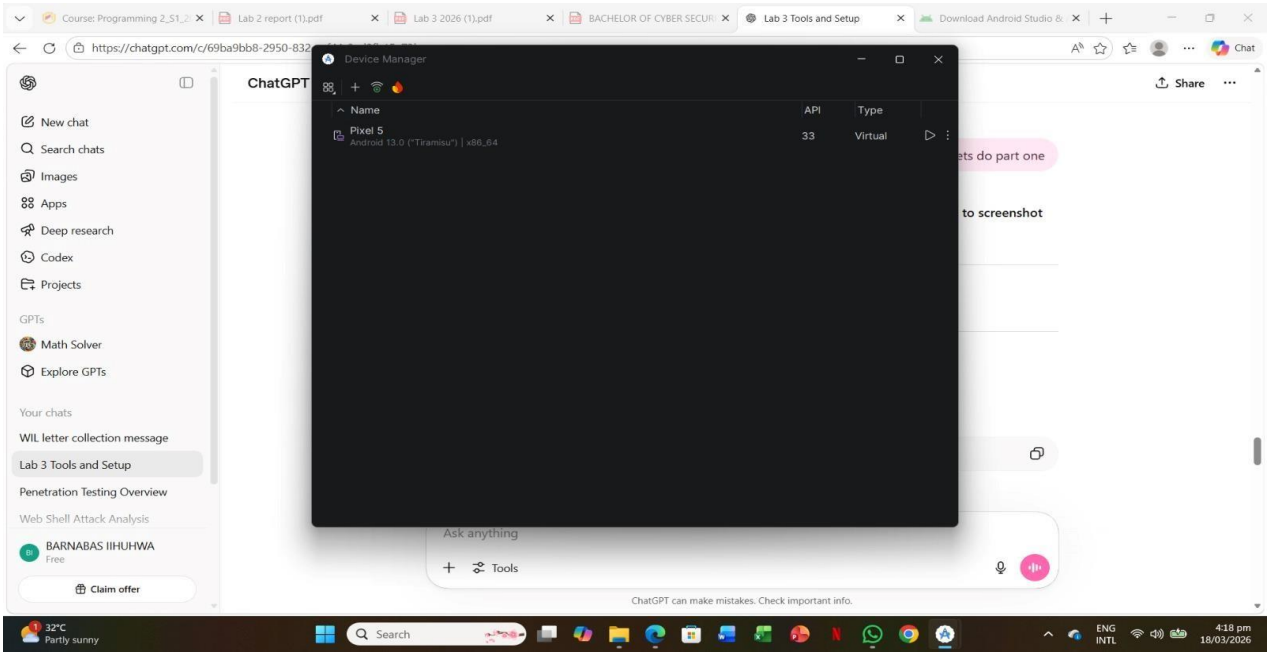
Screenshot System image selection screen:



Step 4: Confirming AVD Creation

After completing the setup, the newly created virtual device appeared in the Virtual Device Manager list, confirming that the AVD was successfully created.

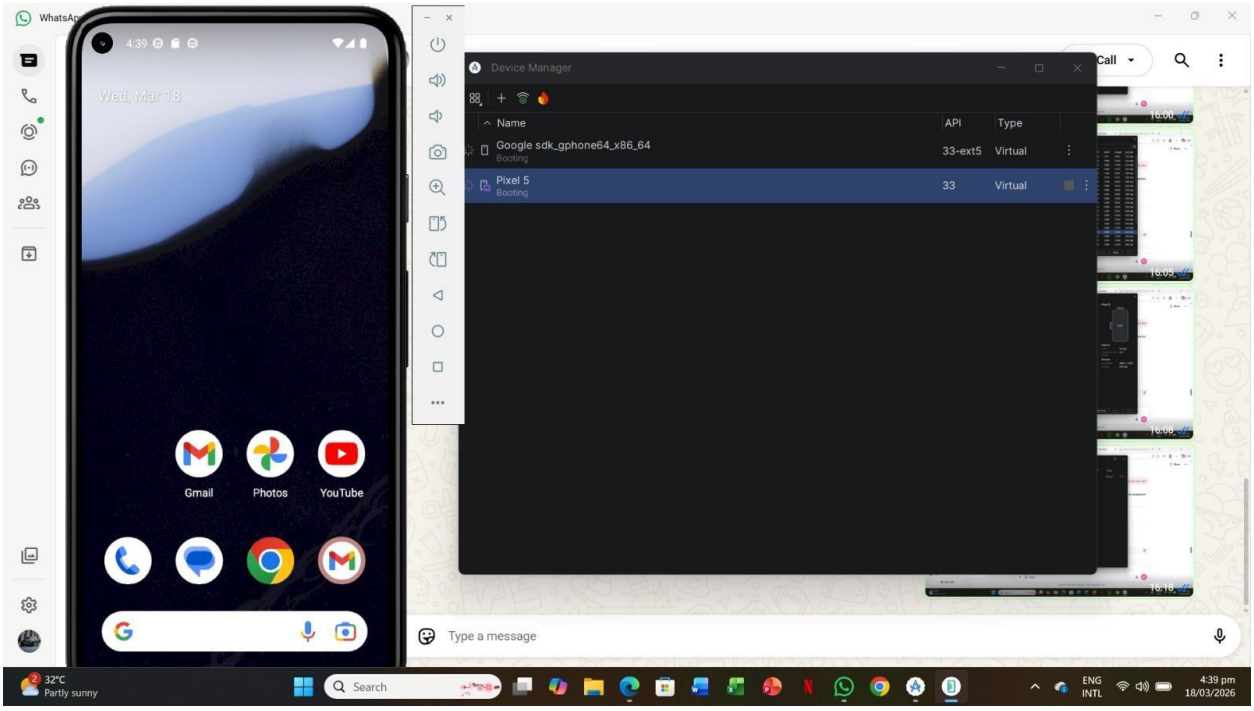
Insert Screenshot AVD listed in Virtual Device Manager



Step 5: Running the Emulator

The created AVD was started by clicking the Play button. The emulator was allowed to fully boot until the Android home screen was visible.

Insert Screenshot Emulator running with home screen visible:

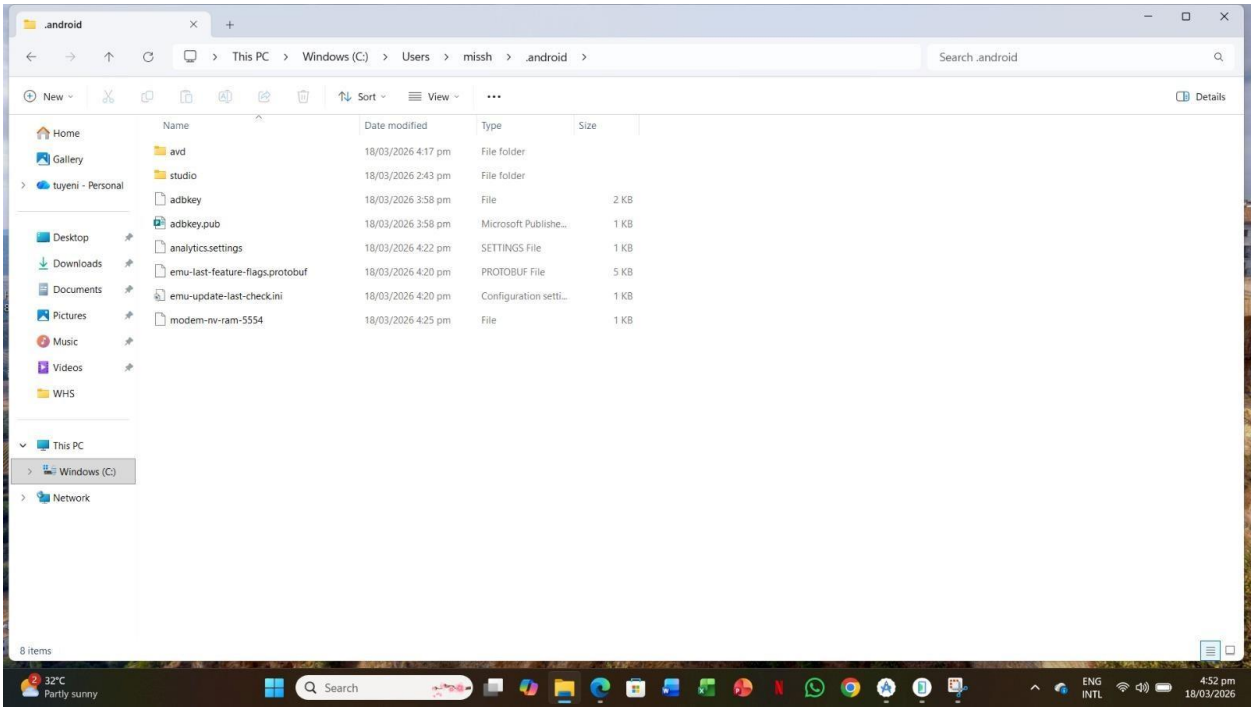


Step 6: Locating the .android Folder

The android directory was located on the computer using the following path: C:\Users\YourName\.android

Hidden files were enabled in File Explorer to ensure the folder was visible. This directory contains configuration files and data related to Android Virtual Devices.

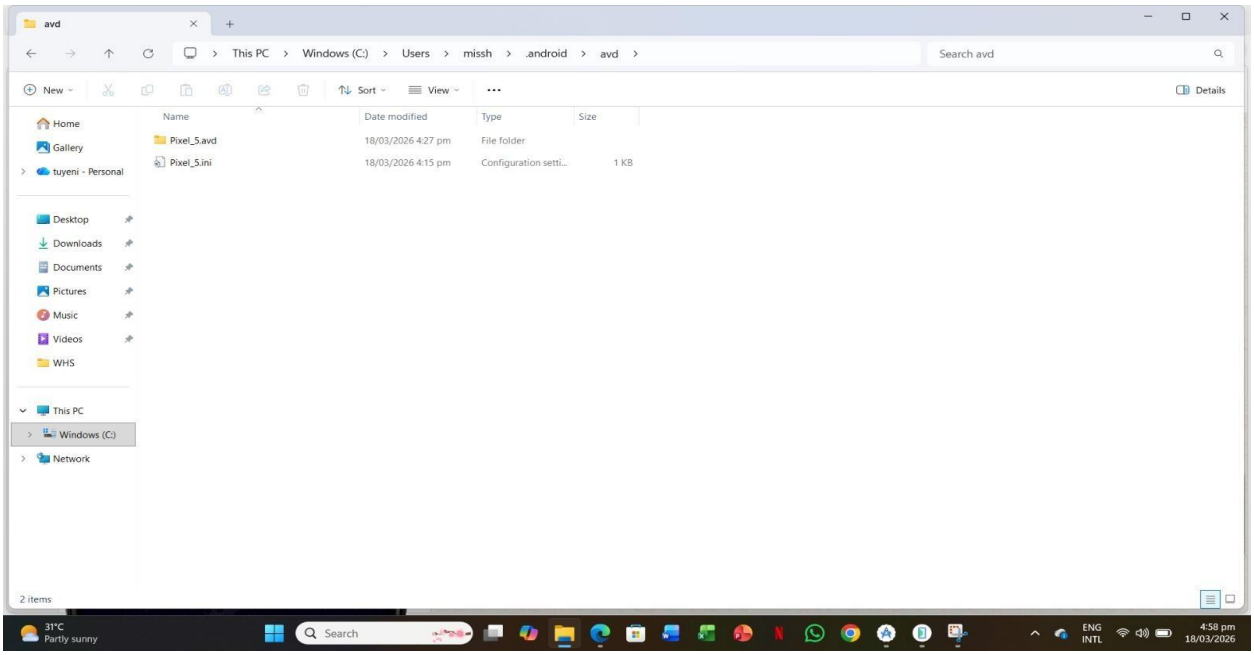
Screenshot android folder opened:



Step 7: Opening the avd Folder

Inside the android directory, the avd folder was opened. This folder contains all created Android Virtual Devices and their associated files.

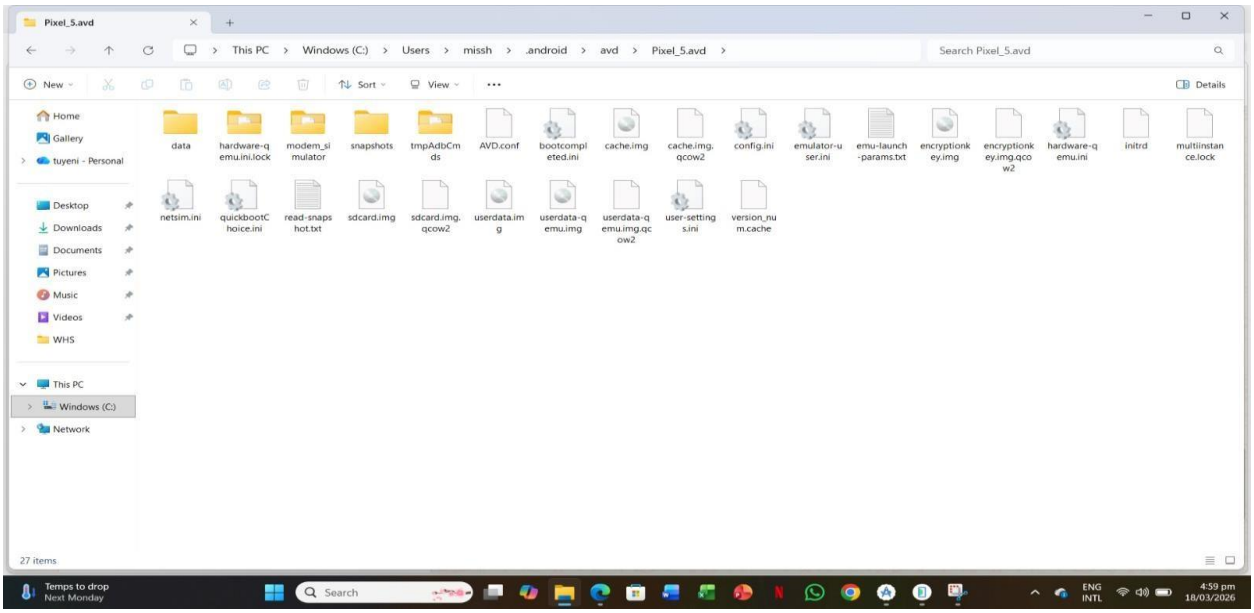
Screenshot Contents of the avd folder:



Step 8: Opening the AVD Folder

The specific folder for the created AVD Pixel_5.avd was opened. This folder contains all files related to that particular virtual device.

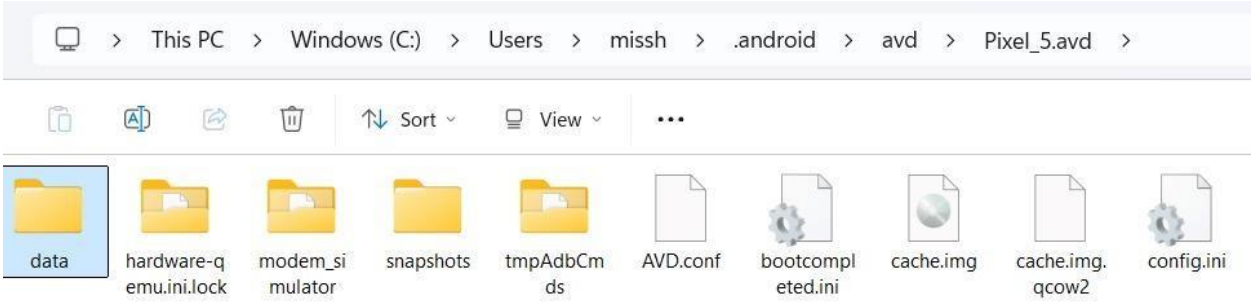
Screenshot Inside the AVD folder showing files:



Step 9: Locating the cache.img File

Within the AVD folder, the file named cache.img was located. This file stores temporary system data and is important in mobile forensics as it may contain recoverable information.

Screenshot cache.img file clearly visible:



PART B: ADB Setup and Device Exploration

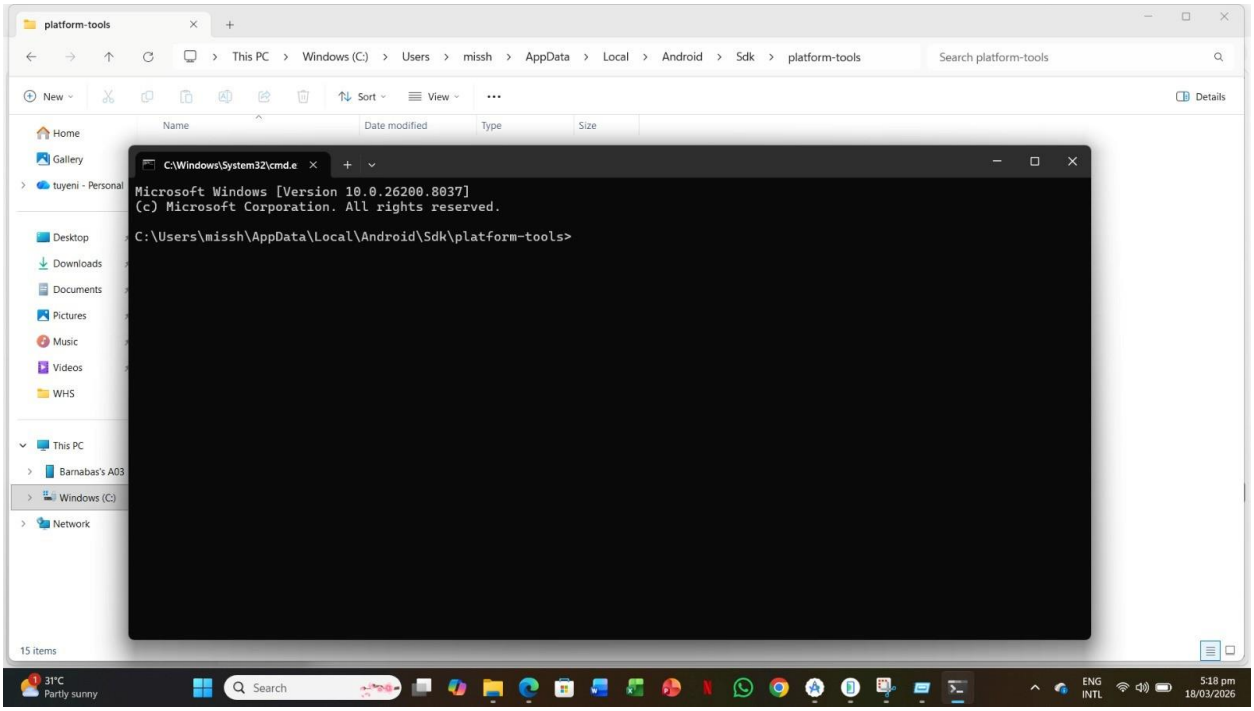
Step 1: Opening the ADB Folder

The folder containing Android Debug Bridge (ADB) was located on the computer. For Android Studio installations, the path used was:

C:\Users\YourName\AppData\Local\Android\Sdk\platform-tools

Inside this folder, adb.exe and other platform tools were present. The command prompt was opened directly from this folder by typing cmd in the path bar and pressing Enter. This allowed execution of ADB commands.

Insert Screenshot Command Prompt opened inside platform-tools folder:



Step 2: Starting the Emulator

The virtual Android device created in Part A was launched to ensure it was running and ready for connection via ADB.

Our Emulator running shown in Part A

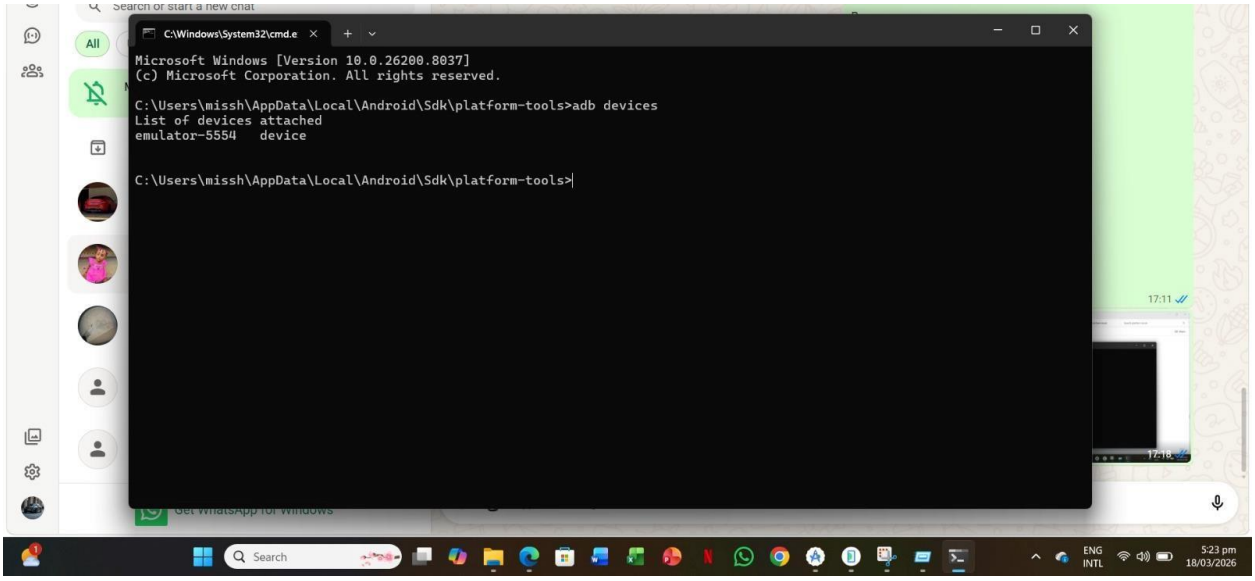
Step 3: Verifying ADB Connection

The connection between ADB and the emulator was confirmed by running the command:

adb devices

The emulator appeared in the output list (e.g., `emulator-5554 device`), confirming that ADB could communicate with the virtual device.

Screenshot Output of adb devices showing the emulator connected:



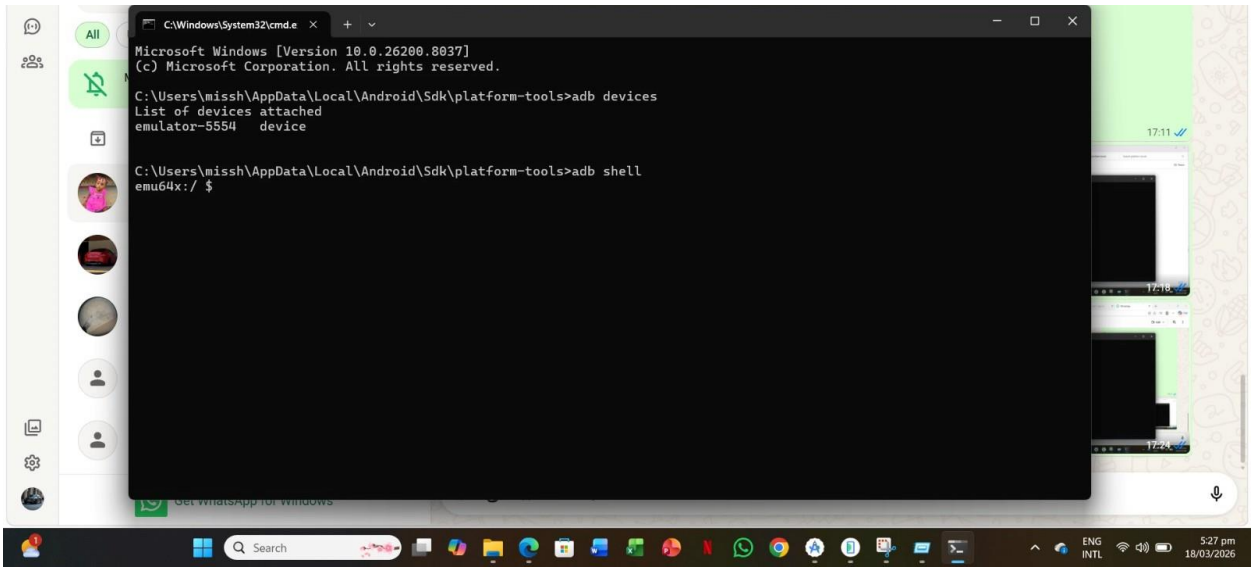
Step 4: Accessing the Device Shell

ADB shell was entered using the command:

```
adb shell
```

This opened a command-line interface inside the Android operating system, indicated by the prompt,

Screenshot Command prompt inside ADB shell:



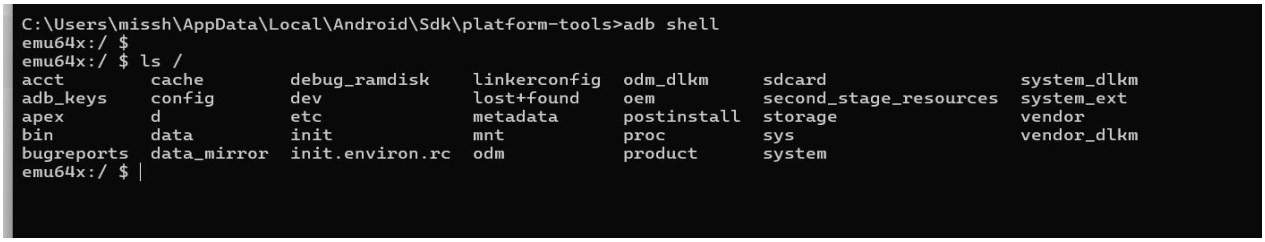
Step 5: Exploring the Root Directory

The root directory of the Android device was explored using:

```
ls /
```

This displayed the main system directories on the device.

Screenshot Root directory contents:



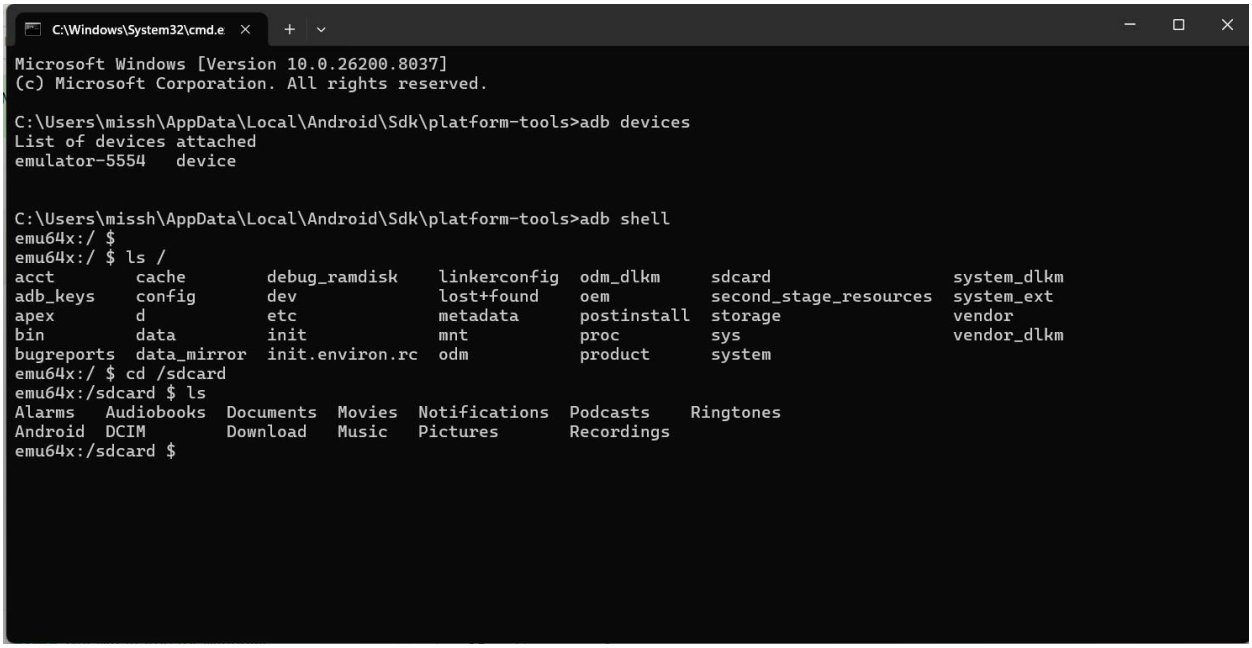
Step 6: Exploring Storage Directory

The SD card storage of the device was explored by running:

```
cd /sdcard
ls
```

This allowed inspection of files stored in user-accessible storage.

Insert Screenshot /sdcard contents:



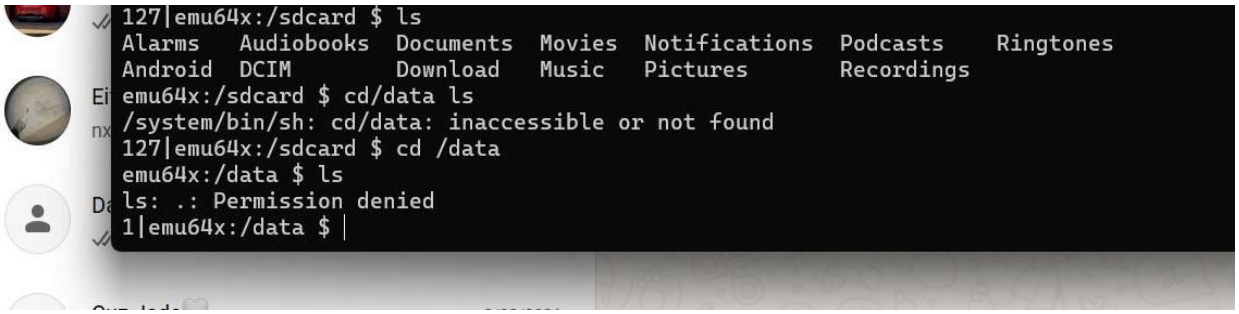
Step 7: Exploring the Data Directory

The /data directory, which stores application and system data, was accessed using:

```
cd /data
ls
```

If permission was denied, this was expected and normal for virtual devices without root access.

Screenshot /data directory contents (with permission denied):



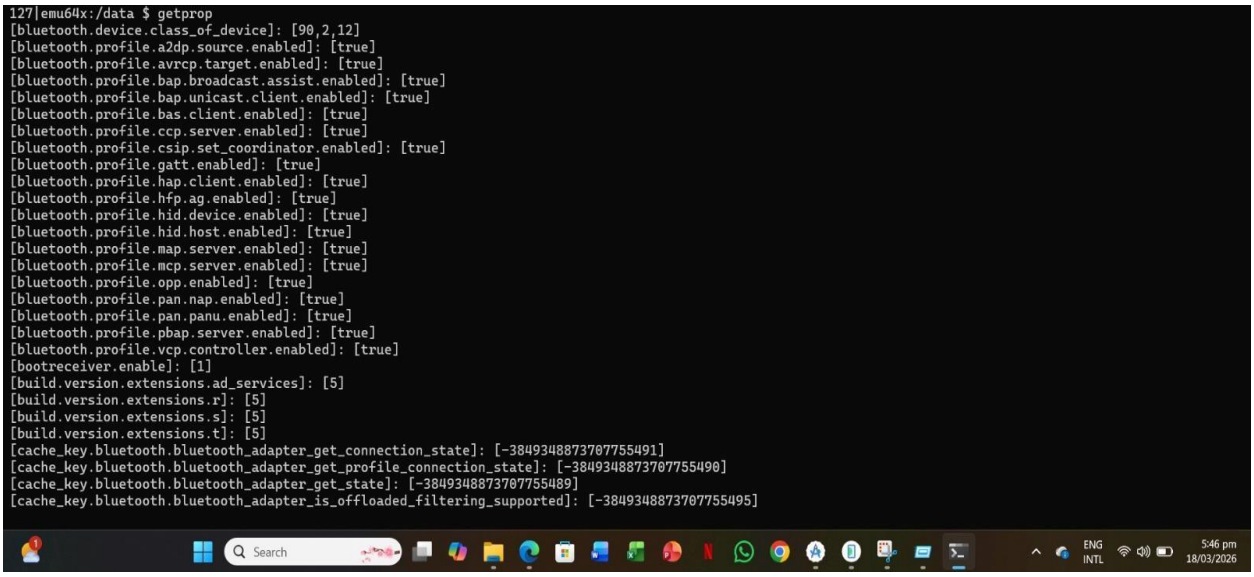
Step 8: Retrieving Device Information

Device properties and system information were obtained with the command:

getprop

This output included system-level details important for forensic analysis.

Screenshot Device properties output:



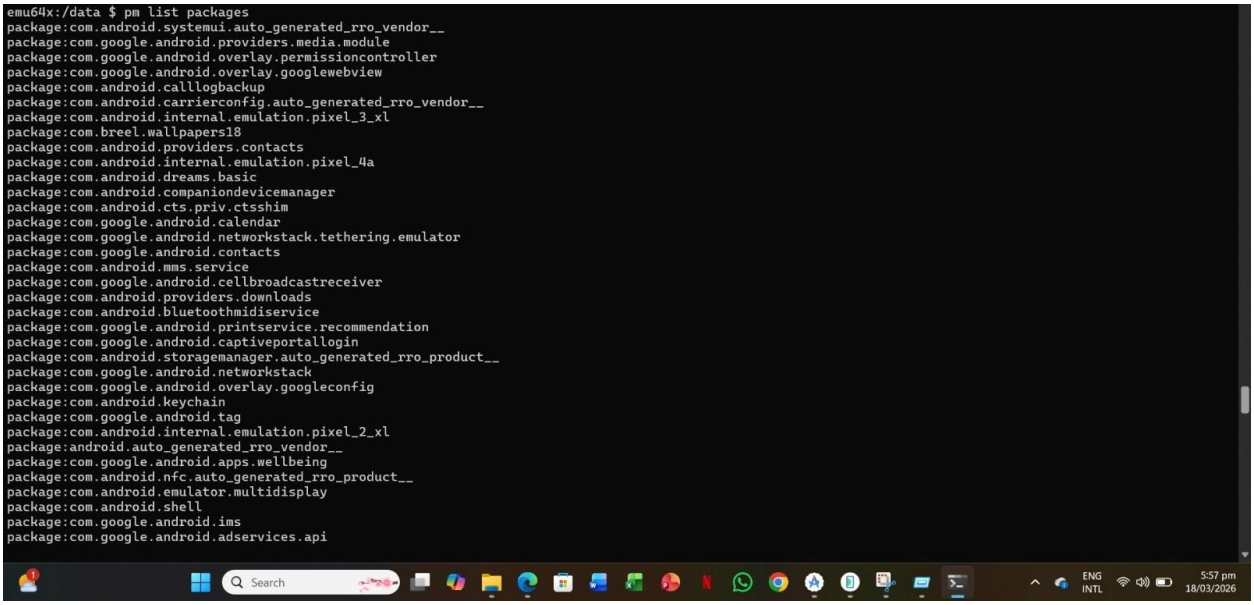
Step 9: Listing Installed Applications

All installed applications on the device were listed using:

pm list packages

This showed packages installed on the virtual Android device, useful in forensic investigations.

Screenshot List of installed apps:



```
emu64x:/data $ pm list packages
package:com.android.systemui.auto_generated_rro_vendor__
package:com.google.android.providers.media.module
package:com.google.android.overlay.permissioncontroller
package:com.google.android.overlay.googlewebview
package:com.android.calllogbackup
package:com.android.carrierconfig.auto_generated_rro_vendor__
package:com.android.internal.emulation.pixel_3_xl
package:com.breel.wallpapers18
package:com.android.providers.contacts
package:com.android.internal.emulation.pixel_4a
package:com.android.dreams.basic
package:com.android.companiondevicemanager
package:com.android.cts.priv.ctsshim
package:com.google.android.calendar
package:com.google.android.networkstack.tethering.emulator
package:com.google.android.contacts
package:com.android.mms.service
package:com.google.android.cellbroadcastreceiver
package:com.android.providers.downloads
package:com.android.bluetoothmidiservice
package:com.google.android.printservice.recommendation
package:com.google.android.captiveportallogin
package:com.android.storagemanager.auto_generated_rro_product__
package:com.google.android.networkstack
package:com.google.android.overlay.googleconfig
package:com.android.keychain
package:com.google.android.tag
package:com.android.internal.emulation.pixel_2_xl
package:android.auto_generated_rro_vendor__
package:com.google.android.apps.wellbeing
package:com.android.nfc.auto_generated_rro_product__
package:com.android.emulator.multidisplay
package:com.android.shell
package:com.google.android.ims
package:com.google.android.adservices.api
```

Step 10: Pulling Data from the Device

Files from the SD card were copied to the host computer using:

adb pull /sdcard/

This extracted data for further forensic analysis.

- Opened ADB shell:

adb shell

- Navigated to emulator storage:

cd /storage/emulated/0/
ls

Verified folders like Download, DCIM, Documents, etc.

- Exited the shell:

exit

- Tried pulling files to PC:

adb pull /storage/emulated/0/Download C:\Users\YourName\Desktop\TestPull

Screenshot File extraction process using adb pull:

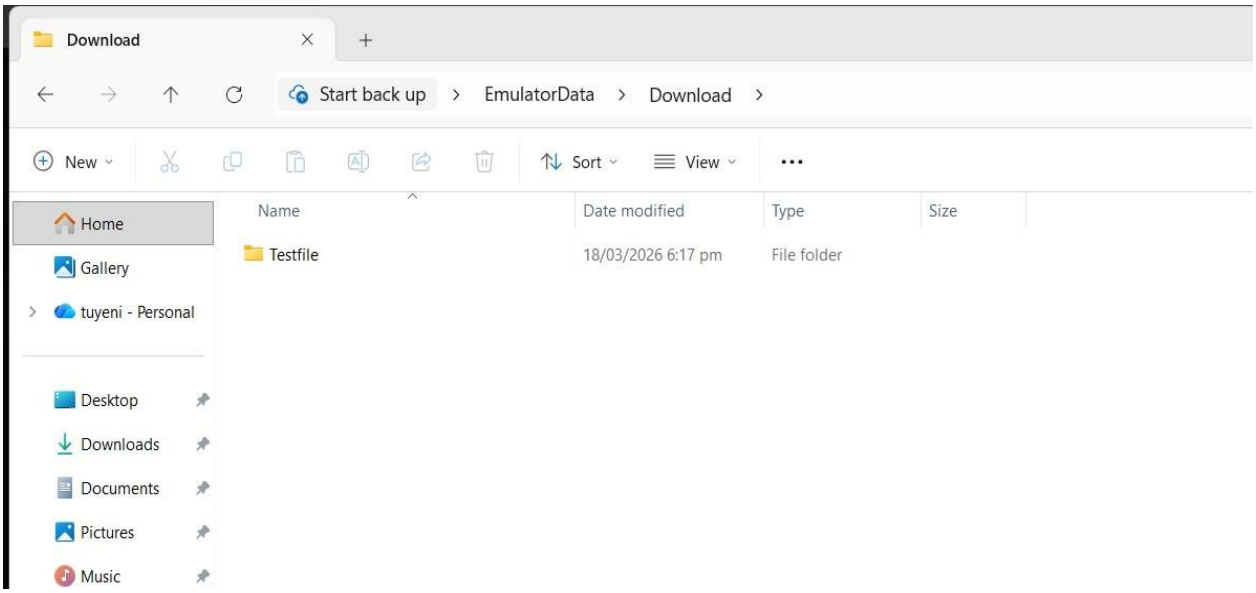
```
emu64x:/data $ pull /sdcard/
/system/bin/sh: pull: inaccessible or not found
127|emu64x:/data $ adb pull /sdcard/
/system/bin/sh: adb: inaccessible or not found
127|emu64x:/data $ adb pull /sdcard/ C:\Users\YourName\Desktop\EmulatorData
/system/bin/sh: adb: inaccessible or not found
127|emu64x:/data $ ls /storage
0000-0000  emulated  self
emu64x:/data $ cd /storage/emulated/0/
emu64x:/storage/emulated/0 $ ls
Alarms  Android  Audiobooks  DCIM  Documents  Download  Movies  Music  Notifications  Pictures  Podcasts  Recordings  Ringtones
emu64x:/storage/emulated/0 $ exit

C:\Users\missh\AppData\Local\Android\Sdk\platform-tools>adb pull /storage/emulated/0/ C:\Users\YourName\Desktop\EmulatorData
adb: error: cannot create file/directory 'C:\Users\YourName\Desktop\EmulatorData': No such file or directory

C:\Users\missh\AppData\Local\Android\Sdk\platform-tools>adb pull /storage/emulated/0/Download C:\Users\YourName\Desktop\TestPull
adb: error: cannot create file/directory 'C:\Users\YourName\Desktop\TestPull': No such file or directory

C:\Users\missh\AppData\Local\Android\Sdk\platform-tools>adb pull /storage/emulated/0/Download C:\Users\YourName\Desktop\Testfile
adb: error: cannot create file/directory 'C:\Users\YourName\Desktop\Testfile': No such file or directory

C:\Users\missh\AppData\Local\Android\Sdk\platform-tools>adb pull /storage/emulated/0/Download C:\Users\missh\Desktop\EmulatorData
/storage/emulated/0/Download/: 0 files pulled, 0 skipped.
```



3. Lab Challenges/Problems Encountered

During the lab, some challenges were encountered. The installation of the Android system image took a long time due to its large size. There were also minor issues with ADB not detecting the device at first, which were resolved by restarting the virtual device and ensuring that ADB was properly installed.

In Step 10, we retrieved files from the emulator's Download folder to the host PC using the adb pull command. We first attempted adb pull /storage/emulated/0/Download

C:\Users\YourName\Desktop\TestPull, but the command failed with the error 'No such file or directory' because the destination folder on the PC did not exist — this was the main challenge in this step. After creating the folder on the PC at C:\Users\missh\Desktop\EmulatorData, we retried the command: adb pull /storage/emulated/0/Download

C:\Users\missh\Desktop\EmulatorData, and the files were successfully copied. This step provided evidence that we were able to extract data from the virtual device for forensic analysis.

4. Conclusion

The lab successfully demonstrated how to use forensic tools to extract and analyse data from an Android device. By creating a virtual device and using ADB, it was possible to access and explore system files. This showed how mobile forensic investigations can be performed and how important data can be recovered from a device.

5. The role played by each group member:

- Titus Shaangelo: Created the virtual device, explored directories, captured screenshots, and compiled the report.

- Barnabas iihuhwa 223081701**:** Assisted with ADB setup and execution of commands and helped verify results and contributed to analysis.

. Tjijenda.u 220113009: download the application needed and configured them.

References

Practical Mobile Forensics.
(n.d.). *Forensically Investigate and Analyse iOS, Android and Windows Devices (4th ed.)*. Packt Publishing.

Android Studio.
(n.d.). Available at: https://developer.android.com/studio

Android Debug Bridge.
(n.d.). Available at: https://developer.android.com/studio/command-line/adb

YouTube.
(n.d.). *ADB Setup Tutorial*. Available at: https://www.youtube.com/watch?v=GERlhgCcoBc

YouTube.
(n.d.). *Exploring Android Device Using ADB*. Available at:
https://www.youtube.com/watch?v=45PcU2HIlYI